Technical Design Document (TDD)

Database Implementation Project Template

A Technical Design Document (TDD) explains **how the database solution will be designed and implemented**. It translates the approved Functional Requirements Document (FRD) into a technical blueprint that a developer can use to build, test, and maintain the database.

For a database implementation project, the TDD documents the technical decisions behind the solution, including:

- The selected database platform and development tools
- The entity-relationship design and relational schema
- Tables, attributes, data types, primary keys, and foreign keys
- Relationship cardinalities and optionality
- Normalization decisions
- Integrity constraints and business-rule enforcement
- Indexes, views, queries, stored procedures, functions, or triggers
- Test strategy, sample data, and security assumptions

The FRD states **what the system must do**. The TDD states **how the database will meet those requirements**.

Functional requirement	Technical design response
The system shall allow staff to enroll a student in a class.	Create an `Enrollment` junction table with `StudentID` and `ClassID` foreign keys.
The system shall prevent duplicate enrollments.	Add a `UNIQUE (StudentID, ClassID)` constraint to `Enrollment`.
The system shall display class capacity and available spaces.	Store capacity in `Class`; use a query or view that counts enrollment records and calculates remaining spaces.

Writing guidance

A good TDD is precise enough that another student could implement the project without guessing important design decisions.

- Use names consistently. If the ERD uses `Customer`, do not call it `Client` in the schema or SQL script.
- Identify every table's primary key, every foreign key, and the reason for important constraints.
- State cardinality clearly, such as "one customer may place many orders" or "each order belongs to exactly one customer."
- Explain why junction tables are needed to resolve many-to-many relationships.
- Show enough technical detail to make the later SQL implementation traceable to the functional requirements.
- Separate design from implementation evidence. The TDD may include representative SQL, but the full executable SQL script can be submitted separately if required by the course.

Required sections

1. Project identification

Item	Your information
Project title	[Insert project title]
Prepared by	[Your name]
Course/section	[Course and section]
Date	[Date]
Version	1.0

2. Design overview

Briefly summarize the database solution. Identify the business area, the main data subjects, and the overall technical approach.

Template

> This project implements a relational database for [organization/business area]. The database will manage [main data subjects]. It will use [DBMS/platform] and will support [major transactions, reports, or processes]. The design consists of approximately [number] related tables and enforces data integrity through primary keys, foreign keys, unique constraints, check constraints, and/or database procedures.

3. Technical environment

Document the technologies, tools, and assumptions used to build and test the project.

Template

Component	Selected technology/tool	Purpose
Database management system	[Example: MySQL 8.x]	Store and manage relational data
Database client/IDE	[Example: MySQL Workbench]	Design, execute, and test SQL
Modeling tool	[Example: draw.io, MySQL Workbench EER]	Create ERD and schema diagram
Source control, if used	[Example: GitHub]	Version SQL scripts and documentation
Operating environment	[Example: Windows/macOS/Linux]	Development and testing environment

4. Architecture and scope

Describe the architecture at the level appropriate for the project. A typical class project can use a simple database-centered architecture.

Template

> The solution uses a relational database architecture. Users or a future application interface submit data-entry and reporting requests to the database. The database stores normalized records, enforces referential integrity, and returns query results. A user interface is outside the scope unless specifically required.

List any architecture or scope choices that affect the design:

- The project uses a single MySQL database/schema.
- The project uses fictional, non-sensitive sample data.
- The project focuses on database implementation; a website or mobile interface is not included.
- Authentication, production backup schedules, and enterprise-scale performance tuning are outside the scope unless required.

5. Entity-relationship design

Describe the key entities and relationships identified from the FRD. Attach or insert the ERD in the final version of the document if your instructor requires one.

Entity summary template

Entity	Purpose	Primary key	Important relationships
[Entity name]	[What the entity represents]	[Primary key]	[Relationships to other entities]
[Entity name]	[What the entity represents]	[Primary key]	[Relationships to other entities]
[Junction entity]	[Resolves many-to-many relationship]	[Primary/composite key]	[Links two parent entities]

Relationship narrative template

- One [Entity A] may be associated with zero, one, or many [Entity B] records.
- Each [Entity B] record must be associated with exactly one [Entity A] record.
- [Entity C] resolves the many-to-many relationship between [Entity A] and [Entity B].

6. Relational schema

Present the implementation-level schema. Include table names, primary keys, foreign keys, and important unique constraints.

Schema notation template

```
TableName(
  PrimaryKey PK,
  AttributeName,
  ForeignKey FK -> ReferencedTable(ReferencedColumn),
  UNIQUE(AttributeName)
)
```

Example format

```
Customer(
  CustomerID PK,
  FirstName,
  LastName,
  Email UNIQUE
)

OrderHeader(
  OrderID PK,
  CustomerID FK -> Customer(CustomerID),
  OrderDate,
```

```
  OrderStatus
)
```

7. Table and attribute specifications

Document each table in detail. Describe every column sufficiently for implementation and testing.

Table specification template

Table: [TableName]

Purpose: [Why the table exists and what one row represents.]

Column	Data type	Null allowed?	Key/constraint	Description/example
[ColumnName]	[Data type]	Yes/No	PK/FK/UNIQUE/CHECK	[Meaning and example]
[ColumnName]	[Data type]	Yes/No	[Constraint]	[Meaning and example]

Rules implemented by this table:

- [Relevant functional requirement or business rule]
- [Relevant functional requirement or business rule]

Repeat this subsection for every table.

8. Normalization analysis

Explain how the design avoids redundant data and common update problems. Most college-level database projects should normally be designed to at least Third Normal Form (3NF), unless there is a documented reason not to do so.

Template

Normal form	Design check	Evidence in this project
First Normal Form (1NF)	Columns contain atomic values; no repeating groups	[Explain]
Second Normal Form (2NF)	Non-key attributes depend on the entire primary key	[Explain]
Third Normal Form (3NF)	Non-key attributes do not depend on other non-key attributes	[Explain]

Useful explanation pattern

[Attribute] is stored in [Table] rather than [OtherTable] because it describes [entity]. This avoids repeating the same information in multiple rows and prevents update anomalies.

9. Integrity, validation, and business-rule enforcement

Map business rules from the FRD to technical controls. This is one of the most important TDD sections because it demonstrates that the database design protects data quality.

Template

Business rule ID	Rule	Technical implementation	Verification method
BR-01	[Business rule]	[PK, FK, UNIQUE, CHECK, trigger, procedure, query logic]	[Test to demonstrate enforcement]
BR-02	[Business rule]	[PK, FK, UNIQUE, CHECK, trigger, procedure, query logic]	[Test to demonstrate enforcement]

10. Database objects and SQL design

List additional database objects that support requirements. Include only objects actually required by the project.

Object type	Object name	Purpose	Related requirements
View	[View name]	[Simplifies a frequently used query]	[FR/RQ IDs]
Stored procedure	[Procedure name]	[Controls a business transaction]	[FR/RQ IDs]
Trigger	[Trigger name]	[Enforces a rule automatically]	[FR/RQ IDs]
Index	[Index name]	[Supports common searches or joins]	[FR/RQ IDs]

11. Query and report design

Define the SQL queries or reports that will demonstrate the system's usefulness. Map them to requirements and identify the tables, joins, filters, grouping, and calculations involved.

Template

Query ID	Query/report	Main tables	Key SQL features	Expected result

RQ-01	[Name]	[Tables]	[JOIN, WHERE, GROUP BY, aggregate, subquery]	[Output]
RQ-02	[Name]	[Tables]	[JOIN, WHERE, GROUP BY, aggregate, subquery]	[Output]

12. Security, privacy, and backup assumptions

For a class project, describe realistic controls while clearly identifying what is simulated or out of scope.

Template

- Access to the database will be limited to authorized project users through database credentials.
- The project will use fictional sample data and will not include real credit-card numbers, passwords, Social Security numbers, or other highly sensitive information.
- In a production environment, the database would use role-based permissions, scheduled backups, encrypted connections, audit logging, and a documented recovery process.
- Production-grade security and backup automation are outside the scope of this course implementation unless specifically required.

13. Test plan and acceptance criteria

Describe the test cases that will prove the technical design meets the FRD. Include both successful operations and tests that should fail because a constraint correctly blocks invalid data.

Template

Test ID	Requirement/rule	Test action	Expected result	Actual result/pass-fail
T-01	[FR/BR ID]	[Action]	[Expected database behavior]	[Complete after testing]
T-02	[FR/BR ID]	[Action]	[Expected database behavior]	[Complete after testing]

14. Requirement traceability

Trace each major functional requirement to one or more technical-design elements.

Template

Functional requirement	Tables/relationships	Constraints or objects	Query/report/test evidence
FR-01	[Tables]	[PK/FK/etc.]	[Query/test]

FR-02	[Tables]	[PK/FK/etc.]	[Query/test]

Worked example: Martial Arts School Management Database

This example continues the Martial Arts School Management Database scenario used in a sample FRD. It shows the type of technical detail expected after functional requirements have been approved.

1. Project identification

Item	Information
Project title	Martial Arts School Management Database
Prepared by	[Student name]
Course/section	[Course and section]
Date	[Date]
Version	1.0

2. Design overview

This project implements a relational MySQL database for a martial arts school. The database manages students, instructors, belt ranks, scheduled classes, student enrollments, attendance, and membership payments. It supports operational tasks such as maintaining student records, assigning instructors to classes, enrolling eligible students, recording attendance and payments, and producing enrollment, attendance, and payment reports.

The design uses seven core tables: `BeltRank`, `Student`, `Instructor`, `Class`, `Enrollment`, `Attendance`, and `Payment`. Primary keys uniquely identify records. Foreign keys enforce valid relationships. A unique constraint prevents duplicate enrollment, check constraints validate allowable values, and a view or procedure can support class-capacity reporting and controlled enrollment.

3. Technical environment

Component	Selected technology/tool	Purpose

Database management system	MySQL 8.x	Stores relational data and executes SQL
Database client/IDE	MySQL Workbench	Creates the schema, runs scripts, and tests queries
Modeling tool	draw.io or MySQL Workbench EER Diagram	Creates the ERD
SQL language	MySQL SQL	Defines tables, constraints, data, and reports
Source control, optional	GitHub	Stores versions of documentation and SQL scripts
Test data	Fictional sample records	Demonstrates functionality without using sensitive data

4. Architecture and scope

The project uses a single-schema relational database architecture. MySQL stores the normalized business data and enforces data integrity. MySQL Workbench is used for database administration and query testing. The system is designed so that a future staff-facing web or desktop application could connect to the database, but application development is outside the scope of this project.

The implementation includes table creation, keys, constraints, sample data, and SQL reports. Real payment processing, staff authentication, instructor payroll, online registration, and a public website are not included.

5. Entity-relationship design

Entity summary

Entity	Purpose	Primary key	Important relationships
BeltRank	Stores the list of recognized belt ranks	BeltRankID	One belt rank may apply to many students and instructors
Student	Stores each school member's personal and membership information	StudentID	One student may have many enrollments and many payments
Instructor	Stores each instructor's contact and employment status information	InstructorID	One instructor may teach many classes
Class	Stores recurring class schedule and capacity information	ClassID	Each class has one instructor; one class may have many enrollments

Enrollment	Resolves the many-to-many relationship between students and classes	EnrollmentID	Each enrollment belongs to one student and one class
Attendance	Stores attendance results for a student's enrollment on a meeting date	AttendanceID	Each attendance record belongs to one enrollment
Payment	Stores membership payments received from a student	PaymentID	Each payment belongs to one student

Relationship narrative

- One BeltRank may be assigned to zero, one, or many Student records. Each student is assigned one belt rank.
- One BeltRank may be assigned to zero, one, or many Instructor records. Each instructor is assigned one belt rank.
- One Instructor may teach zero, one, or many Class records. Each class must have exactly one instructor.
- One Student may have zero, one, or many Enrollment records. Each enrollment belongs to exactly one student.
- One Class may have zero, one, or many Enrollment records. Each enrollment belongs to exactly one class.
- Enrollment resolves the many-to-many relationship between Student and Class.
- One Enrollment may have zero, one, or many Attendance records. Each attendance record belongs to exactly one enrollment.
- One Student may have zero, one, or many Payment records. Each payment belongs to exactly one student.

ERD notation guidance

The completed ERD should show the following relationships:

```
BeltRank 1 ——< Student
BeltRank 1 ——< Instructor
Instructor 1 ——< Class
Student 1 ——< Enrollment >—— 1 Class
Enrollment 1 ——< Attendance
```

```
Student 1 ——< Payment
```

6. Relational schema

```
BeltRank(
   BeltRankID PK,
   RankName UNIQUE,
   RankOrder UNIQUE,
   BeltColor
)

Student(
   StudentID PK,
   FirstName,
   LastName,
   DateOfBirth,
   Phone,
   Email UNIQUE,
   JoinDate,
   BeltRankID FK -> BeltRank(BeltRankID),
   MembershipStatus,
   MembershipExpirationDate
)

Instructor(
   InstructorID PK,
   FirstName,
   LastName,
   Phone,
   Email UNIQUE,
   HireDate,
   BeltRankID FK -> BeltRank(BeltRankID),
   IsActive
)

Class(
   ClassID PK,
   ClassName,
   MartialArtStyle,
   MeetingDay,
   StartTime,
   EndTime,
   Room,
```

```
  Capacity,
  InstructorID FK -> Instructor(InstructorID),
  IsActive
)

Enrollment(
  EnrollmentID PK,
  StudentID FK -> Student(StudentID),
  ClassID FK -> Class(ClassID),
  EnrollmentDate,
  EnrollmentStatus,
  UNIQUE(StudentID, ClassID)
)

Attendance(
  AttendanceID PK,
  EnrollmentID FK -> Enrollment(EnrollmentID),
  AttendanceDate,
  AttendanceStatus,
  UNIQUE(EnrollmentID, AttendanceDate)
)

Payment(
  PaymentID PK,
  StudentID FK -> Student(StudentID),
  PaymentDate,
  Amount,
  PaymentMethod,
  MembershipPeriodStart,
  MembershipPeriodEnd
)
```

7. Table and attribute specifications

Table: BeltRank

Purpose: Stores standardized martial arts belt ranks. One row represents one rank level.

Column	Data type	Null allowed?	Key/constraint	Description/example
BeltRankID	INT	No	PK, auto-increment	Unique identifier for a belt rank; example: 1
RankName	VARCHAR(50)	No	UNIQUE	Rank name; example: White Belt

RankOrder	TINYINT	No	UNIQUE, CHECK > 0	Sequence of rank progression; example: 1
BeltColor	VARCHAR(30)	Yes	None	Belt color; example: White

Rules implemented by this table:

- BR-01: Each rank has a unique identifier.
- Belt rank names and progression order should not be duplicated.

Table: Student

Purpose: Stores one row for each martial arts student/member.

Column	Data type	Null allowed?	Key/constraint	Description/example
StudentID	INT	No	PK, auto-increment	Unique student identifier
FirstName	VARCHAR(50)	No	None	Student first name
LastName	VARCHAR(50)	No	None	Student last name
DateOfBirth	DATE	No	None	Student birth date
Phone	VARCHAR(25)	Yes	None	Contact phone number
Email	VARCHAR(100)	Yes	UNIQUE	Email address, if provided
JoinDate	DATE	No	DEFAULT current date or supplied value	Date the student joined the school
BeltRankID	INT	No	FK -> BeltRank(BeltRankID)	Student's current belt rank
MembershipStatus	VARCHAR(15)	No	CHECK allowed status values	Active, Inactive, Suspended, or Expired
MembershipExpirationDate	DATE	Yes	None	Membership end date

Rules implemented by this table:

- FR-01: Student identity, contact, rank, and membership data are stored.
- BR-09: Membership status is restricted to valid values.

Table: Instructor

Purpose: Stores one row for each instructor who may be assigned to teach classes.

Column	Data type	Null allowed?	Key/constraint	Description/example
InstructorID	INT	No	PK, auto-increment	Unique instructor identifier
FirstName	VARCHAR(50)	No	None	Instructor first name
LastName	VARCHAR(50)	No	None	Instructor last name
Phone	VARCHAR(25)	Yes	None	Contact phone number
Email	VARCHAR(100)	Yes	UNIQUE	Instructor email address
HireDate	DATE	No	None	Date instructor began working at school
BeltRankID	INT	No	FK -> BeltRank(BeltRankID)	Instructor belt rank
IsActive	BOOLEAN	No	DEFAULT TRUE	Indicates whether instructor may be assigned to active classes

Rules implemented by this table:

- FR-02: Instructor information is stored.
- Each instructor's rank must exist in BeltRank.

Table: Class

Purpose: Stores one recurring scheduled martial arts class. One row represents a class offering, such as "Youth Karate — Tuesday 5:30 PM."

Column	Data type	Null allowed?	Key/constraint	Description/example
ClassID	INT	No	PK, auto-increment	Unique class identifier
ClassName	VARCHAR(100)	No	None	Name of class offering
MartialArtStyle	VARCHAR(50)	No	None	Style; example: Karate
MeetingDay	VARCHAR(10)	No	CHECK valid day values	Day of recurring meeting
StartTime	TIME	No	None	Class start time

EndTime	TIME	No	Must be later than StartTime	Class end time
Room	VARCHAR(30)	No	None	Training room or location
Capacity	INT	No	CHECK > 0	Maximum enrollment allowed
InstructorID	INT	No	FK -> Instructor(InstructorID)	Assigned instructor
IsActive	BOOLEAN	No	DEFAULT TRUE	Whether the class is currently offered

Rules implemented by this table:

- FR-03: Class schedule, instructor, and capacity are stored.
- BR-02: Every class is assigned to one instructor.
- BR-10: Start time must occur before end time.

Table: Enrollment

Purpose: Resolves the many-to-many relationship between students and classes. One row represents one student's enrollment in one class.

Column	Data type	Null allowed?	Key/constraint	Description/example
EnrollmentID	INT	No	PK, auto-increment	Unique enrollment identifier
StudentID	INT	No	FK -> Student(StudentID)	Enrolled student
ClassID	INT	No	FK -> Class(ClassID)	Class in which the student is enrolled
EnrollmentDate	DATE	No	None	Date enrollment was created
EnrollmentStatus	VARCHAR(15)	No	CHECK valid status values	Active, Dropped, or Completed

Rules implemented by this table:

- FR-04: Staff can enroll an active student in a class.
- BR-03: Students and classes have a many-to-many relationship.
- FR-05 and BR-04: UNIQUE(StudentID, ClassID) prevents duplicate enrollment.

- FR-06 and BR-06: Capacity is checked through a procedure, trigger, or controlled insertion query.

Table: Attendance

Purpose: Stores attendance results for a student enrolled in a class. One row represents one student's attendance for one meeting date.

Column	Data type	Null allowed?	Key/constraint	Description/example
AttendanceID	INT	No	PK, auto-increment	Unique attendance identifier
EnrollmentID	INT	No	FK -> Enrollment(EnrollmentID)	Enrollment being marked present or absent
AttendanceDate	DATE	No	None	Date of the class meeting
AttendanceStatus	VARCHAR(10)	No	CHECK valid values	Present, Absent, or Excused

Rules implemented by this table:

- FR-07: Staff can record attendance.
- BR-07: Attendance can be entered only for an existing enrollment.
- UNIQUE(EnrollmentID, AttendanceDate) prevents a duplicate attendance mark for the same student enrollment and meeting date.

Table: Payment

Purpose: Stores membership payments received from students. One row represents one payment transaction.

Column	Data type	Null allowed?	Key/constraint	Description/example
PaymentID	INT	No	PK, auto-increment	Unique payment identifier
StudentID	INT	No	FK -> Student(StudentID)	Student making the payment
PaymentDate	DATE	No	None	Date payment was received
Amount	DECIMAL(10,2)	No	CHECK > 0	Payment amount; example: 125.00

PaymentMethod	VARCHAR(20)	No	CHECK valid values	Cash, Card, Check, or Online
MembershipPeriodStart	DATE	No	None	Start of membership period covered
MembershipPeriodEnd	DATE	No	Must be on/after period start	End of membership period covered

Rules implemented by this table:

- FR-08: Payment details are stored.
- BR-08: Payment amount must be greater than zero.
- FR-14: Payment rows support monthly revenue aggregation.

8. Normalization analysis

Normal form	Design check	Evidence in this project
First Normal Form (1NF)	Attributes contain single, atomic values; repeating groups are removed	A student's enrollment in multiple classes is stored as multiple rows in Enrollment, not as a list of classes in Student. A student's multiple payments are stored as rows in Payment.
Second Normal Form (2NF)	Non-key attributes depend on the complete primary key	Enrollment stores facts about the student/class relationship, such as enrollment date and status. Student name belongs only in Student; class name belongs only in Class.
Third Normal Form (3NF)	Non-key attributes do not depend on other non-key attributes	Belt-rank details are stored in BeltRank, not repeated in Student or Instructor. Instructor contact details are stored in Instructor, not copied into Class.

For example, the instructor's name is not stored in the Class table. Class stores only InstructorID. The instructor's name is retrieved by joining Class to Instructor. This prevents inconsistent names if an instructor changes a phone number or corrects a spelling error.

Similarly, student data is not copied into Enrollment, Attendance, or Payment. Those tables refer to the student through keys, reducing duplication and update anomalies.

9. Integrity, validation, and business-rule enforcement

Business rule ID	Rule	Technical implementation	Verification method

BR-01	Every student must have a unique student ID.	StudentID INT PRIMARY KEY AUTO_INCREMENT	Insert several students and confirm each receives a distinct ID.
BR-02	Each class must be assigned to one instructor.	Class.InstructorID NOT NULL foreign key to Instructor	Attempt to insert a class with a nonexistent instructor ID; insertion should fail.
BR-03	A student may enroll in many classes and each class may contain many students.	Enrollment junction table with foreign keys to Student and Class	Insert several enrollment rows for one student and several students for one class.
BR-04	A student may not enroll in the same class more than once.	UNIQUE (StudentID, ClassID) on Enrollment	Attempt a duplicate enrollment insert; it should fail.
BR-05	Only active students may be enrolled.	Enrollment stored procedure, trigger, or application-controlled validation query checks Student.MembershipStatus = 'Active'	Attempt enrollment for an expired student; operation should be rejected by the selected control.
BR-06	Enrollment may not exceed class capacity.	Stored procedure or trigger compares active enrollment count with Class.Capacity before inserting	Fill a class to capacity and attempt one additional enrollment; it should be rejected.
BR-07	Attendance may be recorded only for an enrolled student.	Attendance.EnrollmentID foreign key to Enrollment	Attempt attendance entry with an invalid enrollment ID; insertion should fail.
BR-08	Payment amount must be positive.	CHECK (Amount > 0)	Attempt a payment insert with zero or negative amount; it should fail.
BR-09	Membership status must use approved values.	CHECK (MembershipStatus IN ('Active','Inactive','Suspended','Expired'))	Attempt to insert an unsupported status value; it should fail.
BR-10	Class end time must occur after its start time.	CHECK (EndTime > StartTime) where supported, or trigger/procedure validation	Attempt to create a class ending before it begins; operation should fail.

10. Database objects and SQL design

Object type	Object name	Purpose	Related requirements
View	vw_ClassCapacity	Displays each active class with capacity, active enrollment count, and remaining spaces	FR-06, FR-13
View	vw_StudentPaymentStatus	Combines student membership status with most recent payment information	FR-11
Stored procedure	sp_EnrollStudent	Checks active membership and available capacity before adding an enrollment	FR-04, FR-05, FR-06
Index	idx_Enrollment_ClassID	Supports roster and capacity queries by class	FR-10, FR-13
Index	idx_Attendance_Date	Supports attendance reports filtered by date range	FR-12
Index	idx_Payment_PaymentDate	Supports monthly payment summaries	FR-14

Representative SQL: duplicate enrollment prevention

```
CREATE TABLE Enrollment (
  EnrollmentID INT AUTO_INCREMENT PRIMARY KEY,
  StudentID INT NOT NULL,
  ClassID INT NOT NULL,
  EnrollmentDate DATE NOT NULL,
  EnrollmentStatus VARCHAR(15) NOT NULL DEFAULT 'Active',
  CONSTRAINT uq_enrollment_student_class UNIQUE (StudentID, ClassID),
  CONSTRAINT fk_enrollment_student
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  CONSTRAINT fk_enrollment_class
    FOREIGN KEY (ClassID) REFERENCES Class(ClassID),
  CONSTRAINT chk_enrollment_status
    CHECK (EnrollmentStatus IN ('Active', 'Dropped', 'Completed'))
);
```

Representative SQL: capacity view

```
CREATE VIEW vw_ClassCapacity AS
SELECT
  c.ClassID,
  c.ClassName,
```

```
    c.MeetingDay,
    c.StartTime,
    c.Capacity,
    COUNT(e.EnrollmentID) AS ActiveEnrollmentCount,
    c.Capacity - COUNT(e.EnrollmentID) AS RemainingSpaces
FROM Class AS c
LEFT JOIN Enrollment AS e
    ON c.ClassID = e.ClassID
    AND e.EnrollmentStatus = 'Active'
WHERE c.IsActive = TRUE
GROUP BY
    c.ClassID,
    c.ClassName,
    c.MeetingDay,
    c.StartTime,
    c.Capacity;
```

Representative SQL: controlled enrollment procedure

```
DELIMITER //

CREATE PROCEDURE sp_EnrollStudent (
    IN p_StudentID INT,
    IN p_ClassID INT,
    IN p_EnrollmentDate DATE
)
BEGIN
    DECLARE v_Status VARCHAR(15);
    DECLARE v_Capacity INT;
    DECLARE v_ActiveEnrollmentCount INT;

    SELECT MembershipStatus
    INTO v_Status
    FROM Student
    WHERE StudentID = p_StudentID;

    IF v_Status IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Student does not exist.';
    END IF;

    IF v_Status <> 'Active' THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Only active students may enroll.';
    END IF;

    SELECT Capacity
    INTO v_Capacity
    FROM Class
    WHERE ClassID = p_ClassID AND IsActive = TRUE;

    IF v_Capacity IS NULL THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Active class does not exist.';
    END IF;

    SELECT COUNT(*)
    INTO v_ActiveEnrollmentCount
    FROM Enrollment
    WHERE ClassID = p_ClassID
     AND EnrollmentStatus = 'Active';

    IF v_ActiveEnrollmentCount >= v_Capacity THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Class has reached capacity.';
    END IF;

    INSERT INTO Enrollment (
      StudentID,
      ClassID,
      EnrollmentDate,
      EnrollmentStatus
    )
    VALUES (
      p_StudentID,
      p_ClassID,
      p_EnrollmentDate,
      'Active'
    );
END //

DELIMITER ;
```

Note: The course requirements may not require stored procedures or triggers. If they are not required, students can document a query-based validation approach instead. However, the design must still explain how critical rules—such as duplicate enrollment and class capacity—will be checked.

11. Query and report design

Query ID	Query/report	Main tables	Key SQL features	Expected result
RQ-01	Active student directory	Student, BeltRank	JOIN, WHERE, ORDER BY	Active students with contact information and current rank
RQ-02	Class roster	Class, Enrollment, Student	Multiple JOINs, WHERE, ORDER BY	Students enrolled in a selected class
RQ-03	Class capacity report	Class, Enrollment	LEFT JOIN, COUNT, GROUP BY, arithmetic calculation	Capacity, active enrollment count, and spaces remaining
RQ-04	Expired/overdue membership report	Student, Payment	LEFT JOIN, MAX, date comparison, grouping	Students whose memberships are expired and most recent payment details
RQ-05	Attendance summary	Student, Enrollment, Class, Attendance	JOIN, conditional aggregation, date filter, GROUP BY	Attendance counts and percentage by student/class
RQ-06	Monthly payment summary	Payment	SUM, COUNT, month/year extraction, GROUP BY	Number of payments and total membership revenue per month
RQ-07	Instructor teaching schedule	Instructor, Class	JOIN, WHERE, ORDER BY	Active instructor schedule by day and time

Example query: class roster

```
SELECT
  c.ClassName,
  c.MeetingDay,
  c.StartTime,
  s.StudentID,
  s.LastName,
  s.FirstName,
  e.EnrollmentDate
FROM Enrollment AS e
JOIN Student AS s
  ON e.StudentID = s.StudentID
JOIN Class AS c
  ON e.ClassID = c.ClassID
```

```
WHERE e.ClassID = 1
  AND e.EnrollmentStatus = 'Active'
ORDER BY s.LastName, s.FirstName;
```

Example query: monthly payment summary

```
SELECT
   YEAR(PaymentDate) AS PaymentYear,
   MONTH(PaymentDate) AS PaymentMonth,
   COUNT(*) AS NumberOfPayments,
   SUM(Amount) AS TotalPaymentsReceived
FROM Payment
GROUP BY YEAR(PaymentDate), MONTH(PaymentDate)
ORDER BY PaymentYear, PaymentMonth;
```

12. Security, privacy, and backup assumptions

- The development database will be accessed only through the student's authorized MySQL account.
- The project will use fictional sample data. It will not store real bank-account details, payment-card numbers, passwords, Social Security numbers, or medical information.
- In a production deployment, front-desk staff, instructors, and managers would receive different database or application roles based on the minimum permissions required to perform their duties.
- A production version would use encrypted connections, scheduled backups, restore testing, audit logging, and a documented retention policy.
- Enterprise security, automated backups, and role-based access configuration are outside the scope of this course implementation.

13. Test plan and acceptance criteria

Test ID	Requirement/rule	Test action	Expected result	Actual result/pass-fail
T-01	FR-01	Insert a student with valid required information and belt rank	Student record is inserted with a unique StudentID	[Complete after testing]
T-02	BR-02	Insert a class with an instructor ID that does not exist	Insert is rejected by the foreign key constraint	[Complete after testing]

T-03	FR-04	Enroll an active student in a class with open capacity	New active enrollment row is inserted	[Complete after testing]
T-04	FR-05 / BR-04	Attempt to enroll the same student in the same class a second time	Insert is rejected by the unique constraint	[Complete after testing]
T-05	BR-05	Attempt to enroll an expired student	Enrollment procedure or validation rejects the request	[Complete after testing]
T-06	FR-06 / BR-06	Fill a class to capacity and attempt one more active enrollment	Additional enrollment is rejected	[Complete after testing]
T-07	FR-07	Insert attendance for a valid enrollment and meeting date	Attendance record is inserted	[Complete after testing]
T-08	BR-07	Insert attendance using an enrollment ID that does not exist	Insert is rejected by the foreign key constraint	[Complete after testing]
T-09	BR-08	Insert a payment with an amount of 0.00 or -25.00	Insert is rejected by the check constraint	[Complete after testing]
T-10	RQ-02	Run the class roster query	Query returns active enrolled students for the selected class	[Complete after testing]
T-11	RQ-03	Query vw_ClassCapacity	Results show capacity, enrollment count, and remaining spaces	[Complete after testing]
T-12	RQ-06	Run the monthly payment summary	Results show grouped payment counts and totals by month	[Complete after testing]

14. Requirement traceability

Functional requirement	Tables/relationships	Constraints or objects	Query/report/test evidence
FR-01: Store student records	Student -> BeltRank	StudentID PK; belt-rank FK; membership-status check	T-01; active student directory
FR-02: Store instructor records	Instructor -> BeltRank	InstructorID PK; belt-rank FK; unique email	Instructor schedule query
FR-03: Store classes and instructor assignments	Class -> Instructor	ClassID PK; InstructorID FK; capacity/time checks	T-02; instructor schedule query

FR-04: Enroll students in classes	Student <-> Enrollment <-> Class	Enrollment foreign keys; sp_EnrollStudent	T-03
FR-05: Prevent duplicate enrollment	Enrollment	UNIQUE(StudentID, ClassID)	T-04
FR-06: Prevent over-capacity enrollment	Class, Enrollment	sp_EnrollStudent; vw_ClassCapacity	T-06; T-11
FR-07: Record attendance	Enrollment -> Attendance	Attendance.EnrollmentID FK; unique enrollment/date pair	T-07; T-08; attendance summary
FR-08: Record payments	Student -> Payment	PaymentID PK; positive-amount check; student FK	T-09; payment records
FR-09: Update membership status	Student	Valid-status check constraint	Student update demonstration
FR-10: Display class roster	Class, Enrollment, Student	Join relationships; enrollment-status filter	T-10; class roster query
FR-11: Display expired/overdue memberships	Student, Payment	Membership date/status fields; payment aggregation	Expired/overdue membership report
FR-12: Attendance totals by date range	Attendance, Enrollment, Student, Class	Date filter; joins; conditional aggregation	Attendance summary query
FR-13: Enrollment count and remaining capacity	Class, Enrollment	vw_ClassCapacity; aggregate count	T-11
FR-14: Monthly payment totals	Payment	Date-based grouping; SUM(Amount)	T-12; monthly payment summary

Student submission checklist

Before submitting the TDD, confirm that it:

- Identifies the project, database platform, and development tools.
- Explains how the technical design implements the functional requirements.
- Includes or references a complete ERD with entities, relationships, cardinality, and keys.
- Provides a relational schema with all primary keys, foreign keys, and important unique constraints.

- Includes a data dictionary/table specification for every table and column.
- Explains how the design satisfies 1NF, 2NF, and 3NF.
- Maps important business rules to enforceable database controls.
- Lists planned views, procedures, triggers, and indexes when applicable.
- Defines report/query designs that use appropriate joins, filtering, and aggregation.
- Includes both positive and negative test cases.
- Traces major functional requirements to schema objects, constraints, reports, and tests.
- Uses table names, attribute names, and terminology consistently across the FRD, ERD, schema, SQL script, and test documentation.